



Hochschule für Technik
und Wirtschaft Berlin

University of Applied Sciences

Datenbanken

Vorlesungsskript für das 3. Semester

Studiengang Internationale Medieninformatik

10. Datenbankskripte am Beispiel von PL/pgSQL – Teil 1

Dozent: M. Sc. Burak Boyaci

Version: 05.01.2026

Wintersemester 25/26

Dieses Skript unterliegt der *Creative Commons License* CC BY 4.0
(<http://creativecommons.org/licenses/by/4.0/deed.de>)



13

PL/pgSQL - Datenbankskripte

Kapitelübersicht

13.1 Einführung PL/pgSQL	1
13.2 Grundlegender Aufbau von PL/pgSQL Blöcken	2
13.3 Strukturen in PL/pgSQL.....	10

13.1 Einführung PL/pgSQL

PL/pgSQL ist eine prozedurale Programmiersprache, die im Datenbankmanagementsystem von PostgreSQL ausführbar ist. Eine prozedurale Programmiersprache ist eine Sprache, die – im Gegensatz zu objektorientierter Programmierung – prozessbezogen läuft. Es wird also ein vorher definierter Prozess abgearbeitet. PL/pgSQL verbindet SQL als Datenbanksprache mit Funktionen klassischer Programmiersprachen wie Loops, Schleifen etc. Damit bietet PL/pgSQL Möglichkeiten einer Automatisierung in Form von skriptbasierten, prozeduralen Abfolgen. PL/pgSQL ist eine eigens für PostgreSQL entwickelte Ableitung des ANSI Standards der von Oracle entwickelten Sprache PL/SQL. Dabei können SQL-Skript in PL/SQL Form auf zwei Arten ausgeführt werden.

- a) Als Eingabe durch den Anwender, die temporär vom DBMS eingelesen und ausgeführt wird
- b) Als Stored Procedure, die dauerhaft in der Datenbank gespeichert wird und den Funktionsumfang der Datenbank somit erweitert

Der Vorteil einer Implementierung von PL/pgSQL in Datenbanksystemen ist, dass PL/pgSQL als prozedurale Datenbanksprache direkt in das Datenbank Kernel integriert wird, sodass PL/pgSQL in der Datenbankumgebung ausgeführt und kompiliert wird. Dadurch lassen sich Performance-Vorteile realisieren. Durch die direkte Integration in das DBMS können außerdem Sicherheitsaspekte stärker berücksichtigt werden, da eine bessere Zugriffskontrolle ermöglicht werden kann. Durch die enge Integration von SQL ist die Syntax leicht zu erlernen. Alternativen zu prozeduralen Sprachen wie PL/pgSQL bilden Datenbankschnittstellen wie JDBC (Java Database Connectivity) oder ODBC (Open Database Connectivity), über die Programmiersprachen direkt auf das Datenbank-system zugreifen können.

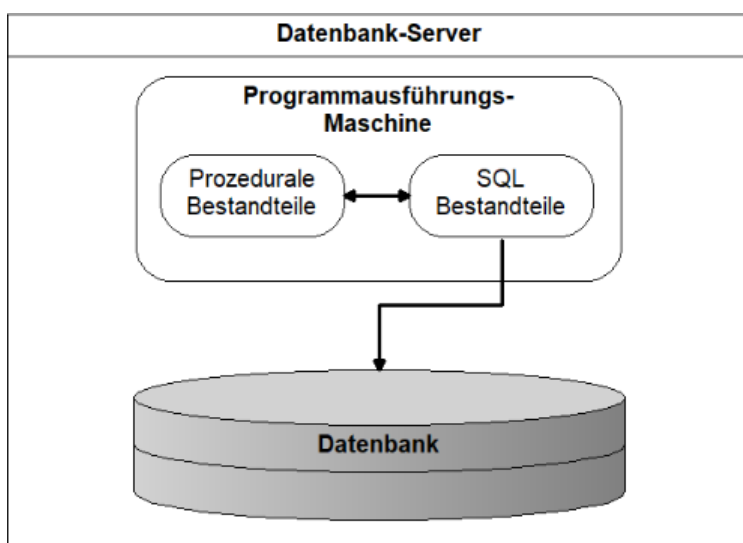


Abbildung 13.1 Architektur - Prozedurale Ausführung von PL/pgSQL in der Datenbankumgebung

13.1 Grundlegender Aufbau von PL/pgSQL Blöcken

Der grundlegende Aufbau in PL/pgSQL erfolgt blockweise. Ein Block ist dabei die Basiseinheit, in der eine PL/pgSQL Anweisung erfolgt. Der typische Block-Aufbau besteht aus folgenden Teilen:

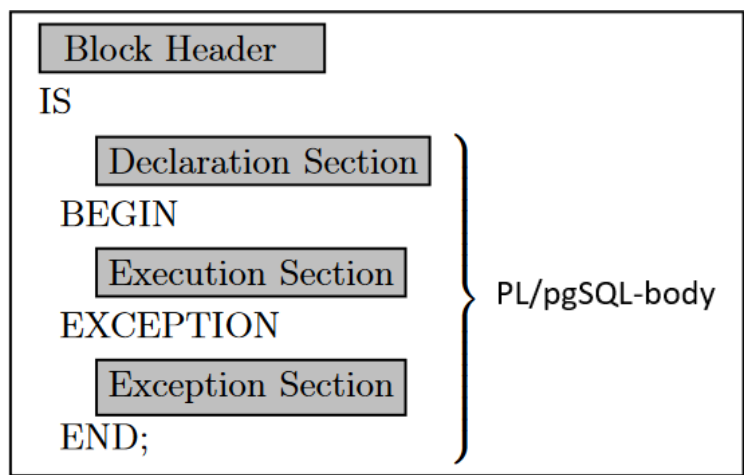


Abbildung 13.2 prinzipieller Aufbau eines anonymen PL/pgSQL Blocks

DECLARE und EXCEPTION Sektion sind dabei optional, während BEGIN und END zwingend sind. Statt IS kann auch AS verwendet werden.

Der Block Header definiert die Art des Schemaobjektes, das beschrieben wird. Dabei können Funktionen, Prozeduren, Trigger oder anonyme Objekte (innerhalb eines anderen Blocks) definiert werden. Zudem werden hier die Aufrufparameter angegeben. Der PL/pgSQL-Body enthält die Definition des Objekts. Dabei werden in der Declaration Section die verwendeten Variablen deklariert. In der Execution Section wird die Befehlssequenz des Blocks angegeben und in der Exception Section werden Reaktionen (exceptions) auf eventuell auftretende Fehlermeldungen definiert. Die PL/pgSQL Blöcke können dabei anonym ausgeführt werden, das heißt sie werden nicht in der Datenbankumgebung gespeichert (Umsetzung mit dem reservierten Wort *declare*) oder können über ein CREATE-Statement in der Datenbankumgebung gespeichert werden. Unabhängige, unbekannte Blöcke werden als *anonyme Blöcke* bezeichnet und sind PL/pgSQL Standard.

Für die nachfolgende Betrachtung von Beispielen sei das folgende Datenmodell gegeben:

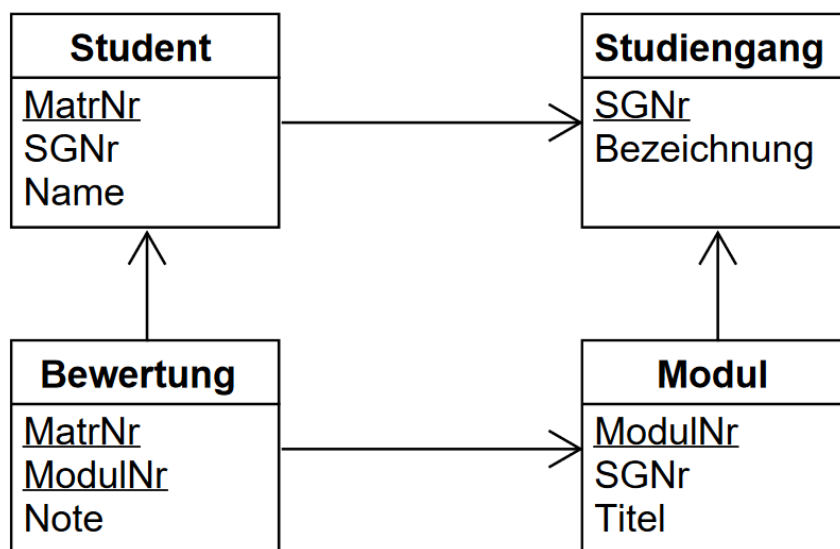


Abbildung 13.3 Datenmodell für Übungsbeispiele

Außerdem ist es sinnvoll, Konventionen der Benennung von Datenbankobjekten vorzunehmen. Damit soll erreicht werden, dass in der Datenbank gespeicherte Tabellen- und Spaltenbezeichnungen von den in PL/pgSQL definierten Variablen unterscheidbar sind.

Folgende Konventionen für die Benennung von Datenbankobjekten vorgesehen werden für die nachfolgenden Beispiele verwenden:

v_	Variablen
t_	Typen
p_	Parameter
cur_	Cursor

Abbildung 13.4 Konventionen der Benennung für PL/pgSQL

Zusätzlich gilt in PL/pgSQL für die Zuweisung von Variablen und bezüglich Gleichheit:

	Java	PL/pgSQL
Zuweisung	=	:=
Gleichheit	==	=

Abbildung 13.5 Vergleich Zuweisung/Gleichheit von Java zu PL/pgSQL

Deklaration eines anonymen Blocks in PL/pgSQL:

```

do $$
declare
  v_matr_nr integer := 501011;
  v_name varchar(25) := 'Meier';
  v_grade decimal(2,1) := 1.3;
begin
  raise notice 'Das ist deine Matrikelnummer: %', v_matr_nr;
  raise notice 'Das ist dein Name: %', v_name;
  raise notice 'Das ist deine Note: %', v_grade;
end $$;
  
```

Ausgabe in PostgreSQL:

```

Data Output  Messages  Notifications
NOTICE:  Das ist deine Matrikelnummer: 501011
NOTICE:  Das ist dein Name: Meier
NOTICE:  Das ist deine Note: 1.3
DO

Query returned successfully in 287 msec.
  
```

Merke: In PL/pgSQL gibt es die Möglichkeit Text über ein 'raise notice' Statement auf der Konsole auszugeben. Außerdem müssen anonyme Blöcke in PL/pgSQL immer mit einem do-Statement begonnen werden. Das Zeichen % ist ein Platzhalter, der bei einer Exception für die Variable in den String implementiert wird und dann in der RAISE Ausgabe mit ausgegeben werden kann. In PL/pgSQL müssen anonyme Blöcke stets mit einem Do-Statement eingeleitet werden.

Es sind für RAISE auch weitere Parameter in PL/pgSQL definiert:

- info
- log
- debug
- warning
- exception

Zur Exception werden wir im EXCEPTION Block des anonymen PL/pgSQL Blocks nähere Informationen erhalten.

Declaration

In der Declaration Sektion des anonymen PL/pgSQL Blocks müssen alle Variablen definiert werden, die im Execution Block benötigt werden. Dabei können Variablen einen beliebigen Datentyp haben. Es sollte aber darauf geachtet werden, die Konventionen der Benennung von Variablen zu berücksichtigen. Die Deklaration von Variablen muss jeweils mit einem Semikolon (;) abgeschlossen werden.

Syntax: *VariablenName* *Datentyp* [*Constraint*] *Zuweisung Wert* ;

DECLARE

v_test text NOT NULL := 'Dies ist ein Test' ;

Query Query History

```
1 do $$
2 declare
3   v_w text := 'Ich';
4   v_x text := 'lerne';
5   v_y text := 'gerne';
6   v_z text := 'Datenbanken.';
7 begin
8   raise notice '% % % %', v_w, v_x, v_y, v_z;
9 end $$;
```

Ausgabe:

Data Output Messages Notifications

```
NOTICE:  Ich lerne gerne Datenbanken.  
DO
```

```
Query returned successfully in 6 secs 157 msec.
```

%Type Attribut

Deklariert eine Variable entsprechend

- a) einer Tabellenspalten-Definition
- b) einer anderen bereits deklarierten Variablen

Syntax:

VariablenName [Name der bereits definierten Tabellenspalte]%type ;

Beispiel:

```
DECLARE  
v_name ClassicModels.customers.customername%type ;
```

- ➔ der neu definierten Variable *v_name* wird die gleiche Formatierung wie der Spalte *customername* aus der Tabelle *customers* zugeordnet

%Rowtype Attribut

Deklariert eine Variable entsprechend einer Menge von Spalten

Syntax:

VariablenName [Name der bereits definierten Tabelle]%Rowtype

Beispiel:

```
DECLARE  
v_product classicmodels.products%rowtype;
```

- ➔ es wird ein Set aus Werten in der Variable *v_product* gespeichert

Beispiel:

Query	Query History
1	<code>do \$\$</code>
2	<code>declare v_product classicmodels.products%rowtype;</code>
3	<code>begin</code>
4	<code>select productname, quantityinstock into v_product</code>
5	<code>from classicmodels.products</code>
6	<code>where quantityinstock > 5000;</code>
7	<code>raise notice 'Die Daten des Kunden lauten: %', v_product;</code>
8	<code>end \$\$;</code>

Ausgabe:

Data Output	Messages	Notifications
NOTICE: Die Daten des Kunden lauten: ("1969 Harley Davidson Ultimate Chopper",7933,,,,,,) DO		
Query returned successfully in 3 secs 400 msec.		

Alternativ kann auch der Datentyp *record* verwendet werden, um einer Variablen ein Set von Werten zuzuordnen:

Query	Query History
1	<code>do \$\$</code>
2	<code>declare</code>
3	<code>v_customer record;</code>
4	<code>begin</code>
5	<code>select customername, creditlimit into v_customer</code>
6	<code>from classicmodels.customers</code>
7	<code>where creditlimit > 100000</code>
8	<code>order by creditlimit desc;</code>
9	<code>raise notice '% ', v_customer;</code>
10	<code>end \$\$;</code>

Ausgabe:

Data Output	Messages	Notifications
NOTICE: ("Euro+ Shopping Channel",227600) DO		
Query returned successfully in 10 secs 658 msec.		

Execution

Die Execution Sektion im anonymen PL/pgSQL Block kann auch dazu benutzt werden Datenbanktabellen zu selektieren oder andere in SQL definierten Operationen auszuführen. An dieser Stelle wird es wichtig die Konventionen, die vorher definiert wurden, zu berücksichtigen.

Query

Query History

```
1 do $$
2 declare v_max_creditlimit int;
3 begin
4 select max(creditlimit) into v_max_creditlimit
5 from ClassicModels.customers;
6 raise notice 'Das maximale Kreditlimit beträgt: %', v_max_creditlimit;
7 end $$;
8
9
```

Data Output

Messages

Notifications

NOTICE: Das maximale Kreditlimit beträgt: 227600
DO

Query returned successfully in 467 msec.

Exception

In der Exception Sektion im anonymen PL/pgSQL Block findet die Fehlerbehandlung anonymen PL/pgSQL Blocks statt. Hier kann definiert werden, wie das Datenbanksystem mit ungewünschtem Input umgehen soll. Es können Kontrollstrukturen in den anonymen Block definiert werden, die greifen, sobald eine unerwünschte Eingabe oder ein Fehler auftritt. Denkbar wäre beispielsweise eine Fehlermeldung bei leeren Datensätzen. In PL/pgSQL erfolgt die Exception erfolgt mit dem RAISE Parameter.

Beispiel:

Wir definieren 2 Variablen v_x und v_y . Wenn v_y kleiner als $v_x \rightarrow$ Ausgabe von v_y und v_x . Wenn jedoch $v_y > v_x \rightarrow$ Fehlermeldung

Query	Query History
1	<code>do \$\$</code>
2	<code>declare</code>
3	<code>v_x int := 2;</code>
4	<code>v_y int := 5;</code>
5	<code>begin</code>
6	<code>if v_y < v_x then</code>
7	<code>raise notice 'Variable_x = %, Variable_y = %', v_x, v_y;</code>
8	<code>elsif v_y >= v_x then</code>
9	<code>raise exception 'Das ist keine gültige Eingabe';</code>
10	<code>end if;</code>
11	<code>end \$\$;</code>

Ausgabe:

Data Output	Messages	Notifications
ERROR: Das ist keine gültige Eingabe		
CONTEXT: PL/pgSQL function inline_code_block line 9 at RAISE		
SQL state: P0001		

Benennung eines anonymen Blocks in PL/pgSQL

```
29 do $Kunde_Name$
30 declare
31 v_customer text;
32 begin
33 select customername into v_customer
34 from classic_cars.customers
35 where customernumber = 103;
36 raise notice 'Der Name des Kunden mit der Kundennummer 103: %', v_customer;
37 end $Kunde_Name$;
```

Data Output	Messages	Notifications
NOTICE: Der Name des Kunden mit der Kundennummer 103: Atelier graphique		
DO		
Query returned successfully in 630 msec.		

\$\$ können benutzt werden, um dem anonymen Block einen Namen zu geben.

13.3 Strukturen in PL/pgSQL

Fallunterscheidungen in PL/pgSQL

Query	Query History
1	do \$\$
2	declare
3	v_x integer;
4	v_y integer;
5	begin
6 ▼	if v_x = 0 then
7	v_y := v_x + 10;
8	end if;
9	end \$\$;

Query	Query History
1	do \$\$
2	declare
3	v_x integer;
4	v_y integer;
5	begin
6 ▼	if v_x = 0 then
7	v_y := v_x + 10;
8	elsif v_x < 0 then
9	v_y := v_x - 20;
10	else
11	v_y := v_x + 20;
12	end if;
13	end \$\$;

Query	Query History
1	do \$\$
2	declare
3	v_x integer;
4	v_y integer;
5	begin
6 ▼	if v_x = 0 then
7	v_y := v_x + 10;
8	else
9	v_y := v_x + 20;
10	end if;
11	end \$\$;

Schleifen in PL/pgSQL

Varianten von Schleifen in PL/pgSQL

Query Query History

```
1  do $$
2  declare
3  v_x integer :=1;
4  v_y integer := 0;
5  begin
6  while v_y <= 10 loop
7    v_x := v_x + v_y;
8    v_y := v_y + 1;
9  end loop;
10 end $$;
```

Query Query History

```
1  do $$
2  declare
3  v_x integer :=1;
4  v_y integer := 0;
5  begin
6  for v_y in 1..5 loop
7    v_x := v_x + v_y;
8  end loop;
9  end $$;
```

Query Query History

```
1  do $$
2  declare
3  v_x integer :=1;
4  v_y integer := 0;
5  begin
6  loop
7    v_x := v_x + v_y;
8    v_y := v_y + 1;
9    if v_y > 3 then
10   exit;
11 end if;
12 end loop;
13 end $$;
```

Beispiel:

Wir wollen folgenden Loop durchführen und eine Benachrichtigung auf der Konsole ausgeben:

Welches Ergebnis liefert folgende Anweisung in PL/pgSQL?

Query Query History

```
1  do $$
2  declare
3  v_x integer :=1;
4  v_y integer := 0;
5  begin
6  loop
7    v_x := v_x + v_y;
8    v_y := v_y + 1;
9    if v_y > 10 then
10   exit;
11  end if;
12  raise notice'Das ist das %. Element des Loops: %', v_y, v_x;
13  end loop;
14  end $$;
```

Ausgabe:

Data Output Messages Notifications

```
NOTICE:  Das ist das 1. Element des Loops: 1
NOTICE:  Das ist das 2. Element des Loops: 2
NOTICE:  Das ist das 3. Element des Loops: 4
NOTICE:  Das ist das 4. Element des Loops: 7
NOTICE:  Das ist das 5. Element des Loops: 11
NOTICE:  Das ist das 6. Element des Loops: 16
NOTICE:  Das ist das 7. Element des Loops: 22
NOTICE:  Das ist das 8. Element des Loops: 29
NOTICE:  Das ist das 9. Element des Loops: 37
NOTICE:  Das ist das 10. Element des Loops: 46
DO
```

Query returned successfully in 93 msec.